

✓ Coursework 2: Image segmentation

In this coursework you will develop and train a convolutional neural network for brain tumour segmentation. Please read both the text and the code in this notebook to get an idea what you are expected to implement. Pay attention to the missing code blocks that look like this:

```
### Insert your code ###  
...  
### End of your code ###
```

What is expected?

- Complete and run the code using `jupyter-lab`.
- Export (File | Save and Export Notebook As...) the notebook as a PDF file, which contains your code, results and answers, and upload the PDF file onto [Scientia](#).
- If Jupyter complains issues during exporting, it is likely that [pandoc](#) or latex is not installed, or their paths have not been included. You can install the relevant libraries and retry. Alternatively, use the Print function of your browser to export the PDF file.
- If Jupyter-lab does not work for you at the end, alternatively, you can use Google Colab to write the code and export the PDF file.

Dependencies

You need to install Jupyter-Lab (https://jupyterlab.readthedocs.io/en/stable/getting_started/installation.html) and other libraries used in this coursework, such as by running the command: `pip3 install [package_name]`

GPU resource

The coursework is developed to be able to run on CPU, as all images have been pre-processed to be 2D and of a smaller size, compared to original 3D volumes.

However, to save training time, you may want to use GPU. In that case, you can run this notebook on Google Colab. On Google Colab, go to the menu, Runtime - Change runtime type, and select **GPU** as the hardware accelerator. At the end, please still export everything and submit as a PDF file on Scientia.

```
# Import libraries  
# These libraries should be sufficient for this tutorial.  
# However, if any other library is needed, please install by yourself.  
import tarfile  
import imageio  
import torch  
import torch.nn as nn  
import torch.nn.functional as F  
import torch.optim as optim  
from torch.utils.data import Dataset  
import numpy as np  
import time  
import os  
import random  
import matplotlib.pyplot as plt  
from matplotlib import colors
```

✓ Q1. Download and visualise the imaging dataset.

The dataset is a public brain imaging dataset from [Medical Decathlon Challenge](#). To save the storage and reduce the computational cost for this tutorial, we extract 2D image slices from the original 3D volumes (T1-Gd contrast enhanced imaging) and downsample the 2D images.

The dataset consists of a training set and a test set. Each image is of dimension 120 x 120, with a corresponding label map of the same dimension. There are four number of classes in the label map:

- 0: background
- 1: edema
- 2: non-enhancing tumour
- 3: enhancing tumour

```
# Download the dataset  
# If you use Ubuntu, wget would natively work.
```

```
# If you use Mac or Windows, which does not have the wget command, you can copy the URL to the web browser and d
# !wget https://www.dropbox.com/s/zmytk2yu284af6t/Task01_BrainTumour_2D.tar.gz

# Unzip the '.tar.gz' file to the current directory
datafile = tarfile.open('Task01_BrainTumour_2D.tar.gz')
datafile.extractall()
datafile.close()

/tmp/ipython-input-2806123473.py:8: DeprecationWarning: Python 3.14 will, by default, filter extracted tar archiv
datafile.extractall()
```

✓ Visualise a random set of 4 training images along with their label maps.

Suggested colour map for brain MR image:

```
cmap = 'gray'
```

Suggested colour map for segmentation map:

```
cmap = colors.ListedColormap(['black', 'green', 'blue', 'red'])
```

```
### Insert your code ###
image_dir = './Task01_BrainTumour_2D/training_images'
label_dir = './Task01_BrainTumour_2D/training_labels'

# Pick 4 random images
random.seed(99)
all_names = os.listdir(image_dir)
chosen = random.sample(all_names, 4)

fig, axes = plt.subplots(4, 2, figsize=(6, 12))
cmap_label = colors.ListedColormap(['black', 'green', 'blue', 'red'])

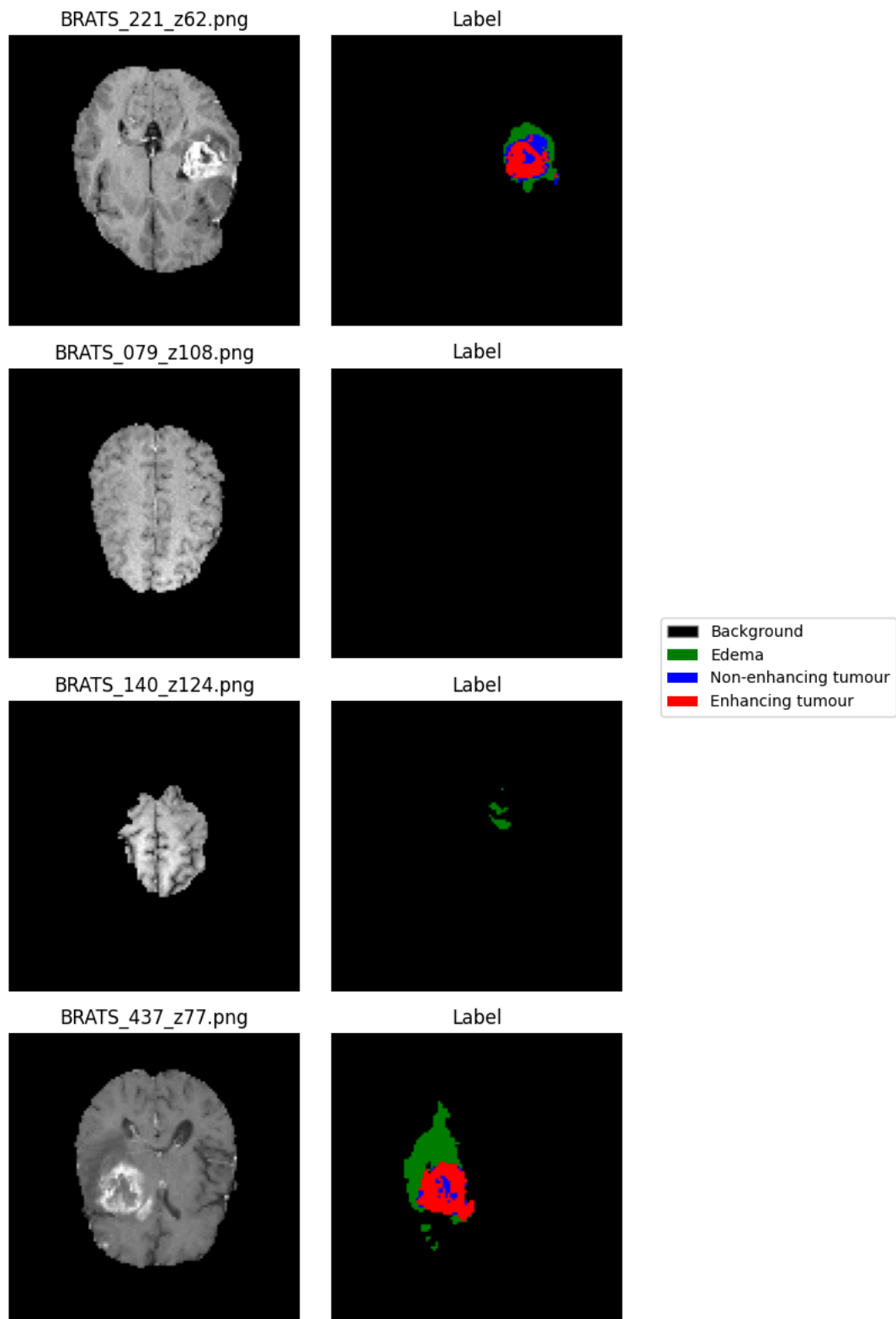
from matplotlib.patches import Patch
legend_elements = [
    Patch(facecolor='black', edgecolor='gray', label='Background'),
    Patch(facecolor='green', label='Edema'),
    Patch(facecolor='blue', label='Non-enhancing tumour'),
    Patch(facecolor='red', label='Enhancing tumour'),
]

for i, name in enumerate(chosen):
    image = imageio.v2.imread(os.path.join(image_dir, name))
    label = imageio.v2.imread(os.path.join(label_dir, name))

    axes[i, 0].imshow(image, cmap='gray')
    axes[i, 0].set_title(name)
    axes[i, 0].axis('off')

    axes[i, 1].imshow(label, cmap=cmap_label, vmin=0, vmax=3)
    axes[i, 1].set_title('Label')
    axes[i, 1].axis('off')

fig.legend(handles=legend_elements, loc='center left', bbox_to_anchor=(1.0, 0.5))
plt.tight_layout(rect=(0, 0, 1, 1))
plt.show()
### End of your code ###
```



Q2. Implement a dataset class.

It can read the imaging dataset and get items, pairs of images and label maps, to be used as training batches.

```
def normalise_intensity(image, thres_roi=1.0):
    """ Normalise the image intensity by the mean and standard deviation """
    # ROI defines the image foreground
    val_l = np.percentile(image, thres_roi)
    roi = (image >= val_l)
    mu, sigma = np.mean(image[roi]), np.std(image[roi])
    eps = 1e-6
    image2 = (image - mu) / (sigma + eps)
    return image2
```

```
class BrainImageSet(Dataset):
```

```

""" Brain image set """
def __init__(self, image_path, label_path='', deploy=False):
    self.image_path = image_path
    self.deploy = deploy
    self.images = []
    self.labels = []

    image_names = sorted(os.listdir(image_path))
    for image_name in image_names:
        # Read the image
        image = imageio.v2.imread(os.path.join(image_path, image_name))
        self.images += [image]

        # Read the label map
        if not self.deploy:
            label_name = os.path.join(label_path, image_name)
            label = imageio.v2.imread(label_name)
            self.labels += [label]

def __len__(self):
    return len(self.images)

def __getitem__(self, idx):
    # Get an image and perform intensity normalisation
    # Dimension: XY
    image = normalise_intensity(self.images[idx])

    # Get its label map
    # Dimension: XY
    label = self.labels[idx]
    return image, label

def get_random_batch(self, batch_size):
    # Get a batch of paired images and label maps
    # Dimension of images: NCXY
    # Dimension of labels: NXY
    images, labels = [], []

    ### Insert your code ###
    indices = random.sample(range(len(self)), batch_size)
    for idx in indices:
        image, label = self[idx] # calls __getitem__
        # Add channel dimension C - shape is (1, X, Y)
        images.append(image[np.newaxis, :, :])
        labels.append(label)

    images = np.array(images)
    labels = np.array(labels)
    ### End of your code ###
    return images, labels

```

✓ Q3. Build a U-net architecture.

Implement a U-net architecture for image segmentation. If you are not familiar with U-net, you can read this paper:

[1] Olaf Ronneberger et al. [U-Net: Convolutional networks for biomedical image segmentation](#). MICCAI, 2015.

For the first convolutional layer, you can start with 16 filters. We have implemented the encoder path. Please complete the decoder path.

```

""" U-net """
class UNet(nn.Module):
    def __init__(self, input_channel=1, output_channel=1, num_filter=16):
        super(UNet, self).__init__()

        # Encoder path
        n = num_filter # 16
        self.conv1 = nn.Sequential(
            nn.Conv2d(input_channel, n, kernel_size=3, padding=1),
            nn.BatchNorm2d(n),
            nn.ReLU(),
            nn.Conv2d(n, n, kernel_size=3, padding=1),
            nn.BatchNorm2d(n),
            nn.ReLU()
        )

        n *= 2 # 32
        self.conv2 = nn.Sequential(
            nn.Conv2d(int(n / 2), n, kernel_size=3, stride=2, padding=1),
            nn.BatchNorm2d(n),

```

```

        nn.ReLU(),
        nn.Conv2d(n, n, kernel_size=3, padding=1),
        nn.BatchNorm2d(n),
        nn.ReLU()
    )

    n *= 2 # 64
    self.conv3 = nn.Sequential(
        nn.Conv2d(int(n / 2), n, kernel_size=3, stride=2, padding=1),
        nn.BatchNorm2d(n),
        nn.ReLU(),
        nn.Conv2d(n, n, kernel_size=3, padding=1),
        nn.BatchNorm2d(n),
        nn.ReLU()
    )

    n *= 2 # 128
    self.conv4 = nn.Sequential(
        nn.Conv2d(int(n / 2), n, kernel_size=3, stride=2, padding=1),
        nn.BatchNorm2d(n),
        nn.ReLU(),
        nn.Conv2d(n, n, kernel_size=3, padding=1),
        nn.BatchNorm2d(n),
        nn.ReLU()
    )

    # Decoder path
    ### Insert your code ###

    # Up-convolve (transposed convolution) output from conv4 layer
    # (128->64 with 2x2 kernel & stride 2),
    # concatenate with skipped connection from conv3 layer (64) -> 128 input,
    # then process to 64 output using 3x3 kernel
    n //= 2 # 64
    self.up_conv1 = nn.ConvTranspose2d(int(n * 2), n, kernel_size=2, stride=2)
    self.conv5 = nn.Sequential(
        nn.Conv2d(int(n * 2), n, kernel_size=3, padding=1),
        nn.BatchNorm2d(n),
        nn.ReLU(),
        nn.Conv2d(n, n, kernel_size=3, padding=1),
        nn.BatchNorm2d(n),
        nn.ReLU()
    )

    n //= 2 # 32
    self.up_conv2 = nn.ConvTranspose2d(int(n * 2), n, kernel_size=2, stride=2)
    self.conv6 = nn.Sequential(
        nn.Conv2d(int(n * 2), n, kernel_size=3, padding=1),
        nn.BatchNorm2d(n),
        nn.ReLU(),
        nn.Conv2d(n, n, kernel_size=3, padding=1),
        nn.BatchNorm2d(n),
        nn.ReLU()
    )

    n //= 2 # 16
    self.up_conv3 = nn.ConvTranspose2d(int(n * 2), n, kernel_size=2, stride=2)
    self.conv7 = nn.Sequential(
        nn.Conv2d(int(n * 2), n, kernel_size=3, padding=1),
        nn.BatchNorm2d(n),
        nn.ReLU(),
        nn.Conv2d(n, n, kernel_size=3, padding=1),
        nn.BatchNorm2d(n),
        nn.ReLU()
    )

    # Final 1x1 convolution to map to output classes
    self.final = nn.Conv2d(n, output_channel, kernel_size=1)
    ### End of your code ###

def forward(self, x):
    # Encoder
    x = self.conv1(x)
    conv1_skip = x

    x = self.conv2(x)
    conv2_skip = x

    x = self.conv3(x)
    conv3_skip = x

    x = self.conv4(x)

```

```

# Decoder
### Insert your code ###
x = self.up_conv1(x)
# concatenate the up-convolution (transposed conv)
# with skip connection from convolutional layer that outputs the same n_channels
x = torch.cat([x, conv3_skip], dim=1)
x = self.conv5(x)

x = self.up_conv2(x)
x = torch.cat([x, conv2_skip], dim=1)
x = self.conv6(x)

x = self.up_conv3(x)
x = torch.cat([x, conv1_skip], dim=1)
x = self.conv7(x)

x = self.final(x)
### End of your code ###
return x

```

✓ Q4. Train the segmentation model.

```

# CUDA device
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print('Device: {}'.format(device))

# Build the model
num_class = 4
model = UNet(input_channel=1, output_channel=num_class, num_filter=16)
model = model.to(device)
params = list(model.parameters())

model_dir = 'saved_models'
if not os.path.exists(model_dir):
    os.makedirs(model_dir)

# Optimizer
optimizer = optim.Adam(params, lr=1e-3)

# Segmentation loss
criterion = nn.CrossEntropyLoss()

# Datasets
train_set = BrainImageSet('Task01_BrainTumour_2D/training_images', 'Task01_BrainTumour_2D/training_labels')
test_set = BrainImageSet('Task01_BrainTumour_2D/test_images', 'Task01_BrainTumour_2D/test_labels')

# Train the model
# Note: when you debug the model, you may reduce the number of iterations or batch size to save time.
num_iter = 10000
train_batch_size = 16
eval_batch_size = 16
start = time.time()
for it in range(1, 1 + num_iter):
    # Set the modules in training mode, which will have effects on certain modules, e.g. dropout or batchnorm.
    start_iter = time.time()
    model.train()

    # Get a batch of images and labels
    images, labels = train_set.get_random_batch(train_batch_size)
    images, labels = torch.from_numpy(images), torch.from_numpy(labels)
    images, labels = images.to(device, dtype=torch.float32), labels.to(device, dtype=torch.long)
    logits = model(images) # forward pass

    # Perform optimisation and print out the training loss
    ### Insert your code ###
    # prevent gradients accumulating across iterations
    optimizer.zero_grad()

    loss = criterion(logits, labels)
    loss.backward()

    if it % 1000 == 0:
        print(f"Training loss (iteration {it}/{num_iter}): {loss}")

    # one step update
    optimizer.step()
    ### End of your code ###

# Evaluate

```

```

if it % 1000 == 0:
    model.eval()
    # Disabling gradient calculation during reference to reduce memory consumption
    with torch.no_grad():
        # Evaluate on a batch of test images and print out the test loss
        ### Insert your code ###
        test_images, test_labels = test_set.get_random_batch(eval_batch_size)
        test_images, test_labels = torch.from_numpy(test_images), \
            torch.from_numpy(test_labels)
        test_images, test_labels = test_images.to(device, dtype=torch.float32), \
            test_labels.to(device, dtype=torch.long)

        test_logits = model(test_images)
        print(f"Evaluation loss (iteration {it}/{num_iter}): {criterion(test_logits, test_labels)}")
        ### End of your code ###

# Save the model
if it % 5000 == 0:
    torch.save(model.state_dict(), os.path.join(model_dir, 'model_{0}.pt'.format(it)))
print('Training took {:.3f}s in total.'.format(time.time() - start))

```

```

Device: cuda
Training loss (iteration 1000/10000): 0.053393058478832245
Evaluation loss (iteration 1000/10000): 0.05420069396495819
Training loss (iteration 2000/10000): 0.026561042293906212
Evaluation loss (iteration 2000/10000): 0.04966496303677559
Training loss (iteration 3000/10000): 0.017305884510278702
Evaluation loss (iteration 3000/10000): 0.024223968386650085
Training loss (iteration 4000/10000): 0.01150582730770111
Evaluation loss (iteration 4000/10000): 0.05361710488796234
Training loss (iteration 5000/10000): 0.014111644588410854
Evaluation loss (iteration 5000/10000): 0.034682098776102066
Training loss (iteration 6000/10000): 0.015387419611215591
Evaluation loss (iteration 6000/10000): 0.03724999725818634
Training loss (iteration 7000/10000): 0.012760507874190807
Evaluation loss (iteration 7000/10000): 0.04997936263680458
Training loss (iteration 8000/10000): 0.010603826493024826
Evaluation loss (iteration 8000/10000): 0.040793780237436295
Training loss (iteration 9000/10000): 0.009585844352841377
Evaluation loss (iteration 9000/10000): 0.04459087550640106
Training loss (iteration 10000/10000): 0.0046248845756053925
Evaluation loss (iteration 10000/10000): 0.059842199087142944
Training took 359.064s in total.

```

Q5. Deploy the trained model to a random set of 4 test images and visualise the automated segmentation.

You can show the images as a 4 x 3 panel. Each row shows one example, with the 3 columns being the test image, automated segmentation and ground truth segmentation.

```

### Insert your code ###
test_image_dir = './Task01_BrainTumour_2D/test_images'
test_label_dir = './Task01_BrainTumour_2D/test_labels'
all_test_names = os.listdir(test_image_dir)

chosen = random.sample(all_test_names, 4)

fig, axes = plt.subplots(4, 3, figsize=(6, 12))

model.eval()

for i, name in enumerate(chosen):
    image = imageio.v2.imread(os.path.join(test_image_dir, name))
    label = imageio.v2.imread(os.path.join(test_label_dir, name))

    image_norm = normalise_intensity(image)
    input_tensor = torch.from_numpy(
        image_norm[np.newaxis, np.newaxis, :, :] # 4D (batched) input for CNN
    ).to(device, dtype=torch.float32)

    with torch.no_grad():
        logits = model(input_tensor)
        pred = torch.argmax(logits, dim=1).squeeze().cpu().numpy()

    axes[i, 0].imshow(image, cmap='gray')
    axes[i, 0].set_title(name)
    axes[i, 0].axis('off')

    axes[i, 1].imshow(pred, cmap=cmap_label, vmin=0, vmax=3)
    axes[i, 1].set_title('Prediction')

```

```

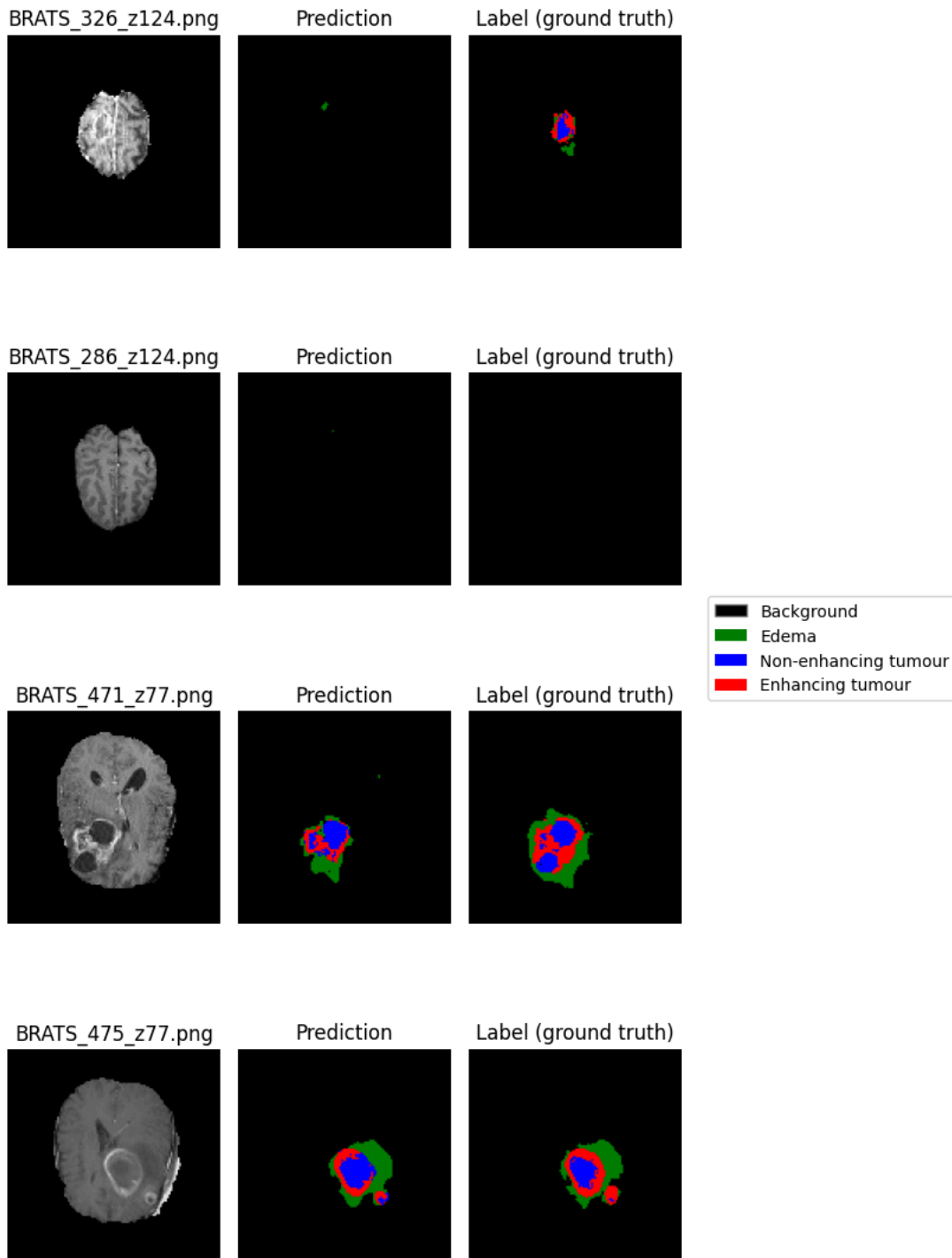
axes[i, 1].axis('off')

axes[i, 2].imshow(label, cmap=cmap_label, vmin=0, vmax=3)
axes[i, 2].set_title('Label (ground truth)')
axes[i, 2].axis('off')

fig.legend(handles=legend_elements, loc='center left', bbox_to_anchor=(1.0, 0.5))
plt.tight_layout(rect=(0, 0, 1, 1))
plt.show()

### End of your code ###

```



Q6. Discussion. Does your trained model work well? How would you improve this model so it can be deployed to the real clinic?

From the 4 test images, the model works fairly well at segmenting large tumours but fails at smaller/more subtle ones.

There's clear overfitting: training loss drops to ~ 0.008 while evaluation loss rises to ~ 0.06 by iteration 10000. Similarly, average loss on the test set is 0.054 (see below). Real-world deployment would need to overcome this. For example, you could use: data augmentation (rotations, flips, elastic deformations), dropout, early stopping, or a larger dataset.

In addition, the model has been trained on a dataset that only contains 3 non-background classes: edema, non-enhancing tumours, and enhancing tumours. So our model would mis-classify any other brain pathologies that it would observe in a real clinic (e.g. aneurysms, strokes). To rectify this, the model should be trained on a dataset containing many different types of pathologies.

Also, the model has been trained on a dataset of almost entirely unhealthy brain scans (i.e. ones that contain a non-background class). This means it's tuned to segment unhealthy brain tissue. This means that the model would likely over-diagnose (many false positives) in a real clinic. To improve this, the model should be trained on a more representative dataset that contains more healthy brain scans.

Another issue is that there is class imbalance: the tumour regions are much smaller than the background. Cross-entropy loss treats all pixels equally, so the model may under-segment small tumour regions (because it can get low-cross entropy by mostly predicting background). Combining cross-entropy with a metric like Dice loss (measuring overlap between predicted and ground truth segmentations) might improve performance.

Also, the model only looks at individual 2D slices of brain scans. This ignores the context of adjacent scans, which are informative for delineating the tumour accurately. A better model would account for this by being trained on adjacent scan slices.

Furthermore, in the real world, MRI scans vary in intensity, contrast, and clarity. The model should be trained on a wider variety of scans from different scanners and hospitals, so it can generalise better to real clinics.

The model lacks interpretability: in the real world it might be useful to see *why* a model is segmenting a scan in a specific way, so that clinicians can understand its reasoning and agree/disagree with it constructively. However this would require a major architectural change.

```
# Evaluate cross-entropy loss on the entire test set
model.eval()
batch_size = 16
total_loss = 0.0
num_batches = 0

with torch.no_grad():
    # Passing in test set in batches of 16
    for i in range(0, len(test_set), batch_size):
        images, labels = [], []
        for j in range(i, min(i + batch_size, len(test_set))):
            image, label = test_set[j]
            images.append(image[np.newaxis, :, :])
            labels.append(label)

        images = torch.from_numpy(np.array(images)).to(device, dtype=torch.float32)
        labels = torch.from_numpy(np.array(labels)).to(device, dtype=torch.long)

        logits = model(images)
        total_loss += criterion(logits, labels).item()
        num_batches += 1

avg_loss = total_loss / num_batches
print(f"Average cross-entropy loss on full test set: {avg_loss:.6f}")
```

Average cross-entropy loss on full test set: 0.051933